Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ЛАБОРАТОРНАЯ 6

По дисциплине Контейнеризация и оркестрация приложений

Тема работы Kubernetes на двух машинах и CI/CD: автоматическая сборка образа и выкатка в кластер

Обучающийся Дощенников Никита Андреевич

Факультет Прикладной информатики

Группа К3321

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Дощенников Н.А.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

Санкт-Петербург
2026

Содержание

Персональные параметры . . . 3
Схема сети . . . 4
Этап 1. Кластер из двух узлов . . . 6
Установка k3s server на master . . . 6
Присоединение worker . . . 6
Проверка кластера . . . 7
Метки узлов . . . 7
Этап 2. CI в GitHub Actions . . . 9
Репозиторий и ветка . . . 9
Workflow `.github/workflows/ci.yml` . . . 9
Запуск пайплайна . . . 10
Публичный образ . . . 12
Этап 3. Развёртывание на двух узлах . . . 13
Изменения в манифестах . . . 13
Применение манифестов . . . 14
Доступ с обеих машин . . . 15
Балансировка . . . 15
Кэш и база данных . . . 16
Проверка APP_OWNER . . . 17
Этап 4. Простой CD: обновление приложения . . . 18
Изменение приложения . . . 18
Rolling update . . . 18
История и откат . . . 19
Этап 5. Отказоустойчивость и финальный артефакт . . . 23
Отключение worker . . . 23
Финальный артефакт . . . 23

Персональные параметры

Параметр	Значение
Номер ИСУ	*465797*
Персональный префикс	*doschennikov*
NodePort приложения	*30097*
Секретный ключ	*secret97doschennikov*
Имя базы данных	*doschennikov97*
Напарник	*this*

Таблица 1. Персональные параметры

Схема сети

Вместо двух физических машин я использовал две виртуальные машины Ubuntu Server 24.04 в virt-manager. Обе ВМ подключены к виртуальной сети `default`. Внутри сети `192.168.122.0/24` ВМ видят друг друга и хост.

Узел	Имя	IP-адрес	Роль
Хост	`virbr0`	192.168.122.1	гипервизор, git, браузер
ВМ №1	`doschennikov-master`	192.168.122.46	control plane, метка `role=db`
ВМ №2	`doschennikov-worker`	192.168.122.217	worker, метка `role=app`

Таблица 2. Узлы кластера

Сеть `default` в libvirt была неактивна, поэтому я её запустил и включил автозапуск:

```
sudo virsh net-list --all
sudo virsh net-start default
sudo virsh net-autostart default
ip -4 addr show virbr0
```

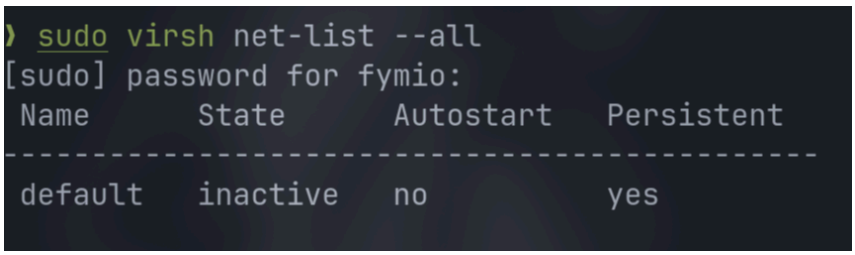


Рисунок 1. Сеть `default` до запуска

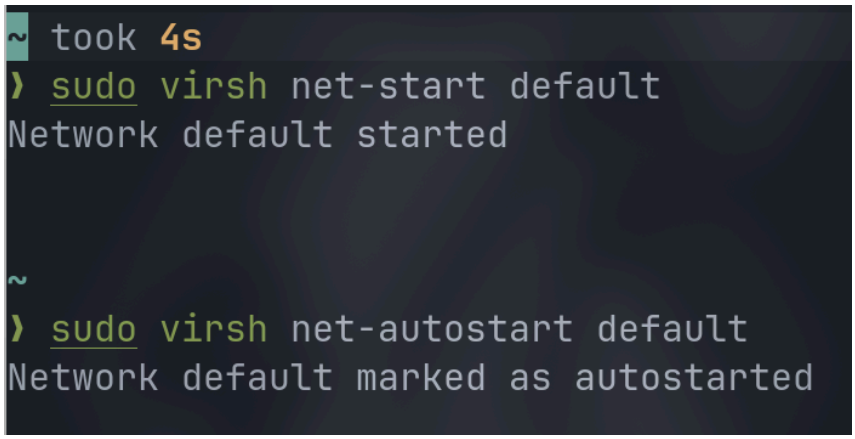


Рисунок 2. Запуск сети `default` и автозапуск

```
) ip -4 addr show virbr0
6: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc htb state DOWN group default qlen 1000
    inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
       valid_lft forever preferred_lft forever
```

Рисунок 3. Адрес моста `virbr0`: 192.168.122.1/24

```
doschennikov-master@doschennikov-master:~$ hostnamectl
 Static hostname: doschennikov-master
       Icon name: computer-vm
         Chassis: vm
      Machine ID: 004207096cf248618a4cc00f0dcbd8bd
         Boot ID: 39043791e22d4330a5930849a5d39665
  Virtualization: kvm
Operating System: Ubuntu 24.04.5 LTS
          Kernel: Linux 6.8.0-139-generic
    Architecture: x86-64
 Hardware Vendor: QEMU
  Hardware Model: Standard PC _Q35 + ICH9, 2009_
Firmware Version: Arch Linux 1.17.0-2-2
   Firmware Date: Tue 2014-04-01
    Firmware Age: 12y 6month 3w 1d
doschennikov-master@doschennikov-master:~$
```

Рисунок 4. `hostnamectl` на master

```
doschennikov-worker@doschennikov-worker:~$ hostnamectl
 Static hostname: doschennikov-worker
       Icon name: computer-vm
         Chassis: vm
      Machine ID: 2e7170b1706d47c589a327e42f85a6c3
         Boot ID: 5c909691731a4e0fadb86703d18aa7af
  Virtualization: kvm
Operating System: Ubuntu 24.04.5 LTS
          Kernel: Linux 6.8.0-139-generic
    Architecture: x86-64
 Hardware Vendor: QEMU
  Hardware Model: Standard PC _Q35 + ICH9, 2009_
Firmware Version: Arch Linux 1.17.0-2-2
   Firmware Date: Tue 2014-04-01
    Firmware Age: 12y 6month 3w 1d
doschennikov-worker@doschennikov-worker:~$
```

Рисунок 5. `hostnamectl` на worker

Этап 1. Кластер из двух узлов

Установка k3s server на master

На первой машине я поставил k3s server, явно указав имя узла и его IP:

```
curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="server --node-name
doschennikov-master --node-ip 192.168.122.46 --tls-san 192.168.122.46"
sh -
sudo k3s kubectl get nodes -o wide
sudo cat /var/lib/rancher/k3s/server/node-token
```

```
doschennikov-master@doschennikov-master:~$ curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="server --node-
name doschennikov-master --node-ip 192.168.122.46 --tls-san 192.168.122.46" sh -
[INFO]  Finding release for channel stable
[INFO]  Using v1.36.5+k3s1 as release
[INFO]  Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.36.5%2Bk3s1/sha256sum-amd64.tx
t
[INFO]  Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.36.5%2Bk3s1/k3s
[INFO]  Verifying binary download
[INFO]  Installing k3s to /usr/local/bin/k3s
[INFO]  Skipping installation of SELinux RPM
[INFO]  Creating /usr/local/bin/kubectl symlink to k3s
[INFO]  Creating /usr/local/bin/crictl symlink to k3s
[INFO]  Creating /usr/local/bin/ctr symlink to k3s
[INFO]  Creating killall script /usr/local/bin/k3s-killall.sh
[INFO]  Creating uninstall script /usr/local/bin/k3s-uninstall.sh
[INFO]  env: Creating environment file /etc/systemd/system/k3s.service.env
[INFO]  systemd: Creating service file /etc/systemd/system/k3s.service
[INFO]  systemd: Enabling k3s unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s.service → /etc/systemd/system/k3s.service.
[INFO]  systemd: Starting k3s
```

Рисунок 6. Установка k3s server на master

```
doschennikov-master@doschennikov-master:~$ sudo k3s kubectl get nodes -o wide
NAME                STATUS   ROLES           AGE   VERSION        INTERNAL-IP      EXTERNAL-IP   OS-IMAG
E            KERNEL-VERSION              CONTAINER-RUNTIME
doschennikov-master   Ready    control-plane   75s   v1.36.5+k3s1   192.168.122.46   <none>        Ubuntu
24.04.5 LTS   6.8.0-139-generic (amd64)   containerd://2.3.4-k3s1.36
```

Рисунок 7. Узел master в состоянии Ready

Токен из `node-token` нужен только для присоединения worker.

Присоединение worker

На второй машине я установил k3s agent, указав адрес API-сервера master и токен:

```
curl -sfL https://get.k3s.io | K3S_URL=https://192.168.122.46:6443
K3S_TOKEN='<k3s_token>' INSTALL_K3S_EXEC="agent --node-name
doschennikov-worker --node-ip 192.168.122.217" sh -
```

```
ea9a7b482d9357a5d' INSTALL_K3S_EXEC="agent --node-name doschennikov-worker --node-ip 192.168.122.217" sh -
[sudo] password for doschennikov-worker:
Sorry, try again.
[sudo] password for doschennikov-worker:
[INFO]  Finding release for channel stable
[INFO]  Using v1.36.5+k3s1 as release
[INFO]  Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.36.5%2Bk3s1/sha256sum-amd64.tx
t
[INFO]  Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.36.5%2Bk3s1/k3s
[INFO]  Verifying binary download
[INFO]  Installing k3s to /usr/local/bin/k3s
[INFO]  Skipping installation of SELinux RPM
[INFO]  Creating /usr/local/bin/kubectl symlink to k3s
[INFO]  Creating /usr/local/bin/crictl symlink to k3s
[INFO]  Creating /usr/local/bin/ctr symlink to k3s
[INFO]  Creating killall script /usr/local/bin/k3s-killall.sh
[INFO]  Creating uninstall script /usr/local/bin/k3s-agent-uninstall.sh
[INFO]  env: Creating environment file /etc/systemd/system/k3s-agent.service.env
[INFO]  systemd: Creating service file /etc/systemd/system/k3s-agent.service
[INFO]  systemd: Enabling k3s-agent unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s-agent.service → /etc/systemd/system/k3s-ag
ent.service.
[INFO]  systemd: Starting k3s-agent
```

Рисунок 8. Установка k3s agent на worker (токен скрыт)

Проверка кластера

```
doschennikov-master@doschennikov-master:~$ sudo k3s kubectl get nodes -o wide
NAME                STATUS   ROLES           AGE     VERSION        INTERNAL-IP       EXTERNAL-IP   OS-I
MAGE             KERNEL-VERSION              CONTAINER-RUNTIME
doschennikov-master   Ready    control-plane   5m19s   v1.36.5+k3s1   192.168.122.46    <none>        Ubun
tu 24.04.5 LTS   6.8.0-139-generic (amd64)   containerd://2.3.4-k3s1.36
doschennikov-worker   Ready    <none>          56s     v1.36.5+k3s1   192.168.122.217   <none>        Ubun
tu 24.04.5 LTS   6.8.0-139-generic (amd64)   containerd://2.3.4-k3s1.36
```

Рисунок 9. Два узла Ready с разными INTERNAL-IP

Для удобства я скопировал kubeconfig и настроил обычный `kubectl`:

```
mkdir -p ~/.kube && sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $USER:$USER ~/.kube/config
export KUBECONFIG=~/.kube/config
kubectl get nodes
```

```
doschennikov-master@doschennikov-master:~$ mkdir -p ~/.kube && sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/c
onfig
sudo chown $USER:$USER ~/.kube/config
export KUBECONFIG=~/.kube/config
echo 'export KUBECONFIG=~/.kube/config' >> ~/.bashrc
kubectl get nodes
NAME                STATUS   ROLES           AGE     VERSION
doschennikov-master   Ready    control-plane   5m55s   v1.36.5+k3s1
doschennikov-worker   Ready    <none>          92s     v1.36.5+k3s1
```

Рисунок 10. Настройка kubeconfig и `kubectl get nodes`

Метки узлов

```
kubectl label node doschennikov-master role=db
kubectl label node doschennikov-worker role=app
kubectl get nodes --show-labels
```

```
doschennikov-master@doschennikov-master:~$ kubectl label node doschennikov-master role=db
kubectl label node doschennikov-worker role=app
kubectl get nodes --show-labels
node/doschennikov-master labeled
node/doschennikov-worker labeled
NAME                 STATUS   ROLES           AGE     VERSION        LABELS
doschennikov-master   Ready    control-plane   6m22s   v1.36.5+k3s1   beta.kubernetes.io/arch=amd64,beta.k
ubernetes.io/instance-type=k3s,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname
=doschennikov-master,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=true,node.kubernetes.io/
instance-type=k3s,role=db
doschennikov-worker   Ready    <none>          119s    v1.36.5+k3s1   beta.kubernetes.io/arch=amd64,beta.k
ubernetes.io/instance-type=k3s,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname
=doschennikov-worker,kubernetes.io/os=linux,node.kubernetes.io/instance-type=k3s,role=app
```

Рисунок 11. Метки `role=db` и `role=app`

```
doschennikov-master@doschennikov-master:~$ kubectl get nodes -o wide
kubectl get nodes --show-labels
NAME                 STATUS   ROLES           AGE     VERSION        INTERNAL-IP       EXTERNAL-IP   OS-I
MAGE             KERNEL-VERSION              CONTAINER-RUNTIME
doschennikov-master   Ready    control-plane   9m57s   v1.36.5+k3s1   192.168.122.46    <none>        Ubun
tu 24.04.5 LTS   6.8.0-139-generic (amd64)   containerd://2.3.4-k3s1.36
doschennikov-worker   Ready    <none>          5m34s   v1.36.5+k3s1   192.168.122.217   <none>        Ubun
tu 24.04.5 LTS   6.8.0-139-generic (amd64)   containerd://2.3.4-k3s1.36
NAME                 STATUS   ROLES           AGE     VERSION        LABELS
doschennikov-master   Ready    control-plane   9m57s   v1.36.5+k3s1   beta.kubernetes.io/arch=amd64,beta.k
ubernetes.io/instance-type=k3s,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname
=doschennikov-master,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=true,node.kubernetes.io/
instance-type=k3s,role=db
doschennikov-worker   Ready    <none>          5m34s   v1.36.5+k3s1   beta.kubernetes.io/arch=amd64,beta.k
ubernetes.io/instance-type=k3s,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname
=doschennikov-worker,kubernetes.io/os=linux,node.kubernetes.io/instance-type=k3s,role=app
doschennikov-master@doschennikov-master:~$
```

Рисунок 12. Итоговое состояние узлов: IP и метки

- Почему PostgreSQL с PersistentVolumeClaim приходится привязывать к одному узлу при storage-классе local-path?

Storage-класс `local-path` создаёт том как обычный каталог на диске конкретного узла (`/var/lib/rancher/k3s/storage/...`). Этот каталог физически существует только на одной машине и не реплицируется на другие узлы. Если под PostgreSQL запустится на другом узле, он не найдёт свои данные и получит пустой том, а сам PersistentVolume по своей привязке к узлу всё равно останется на исходной машине. Поэтому под с БД нужно закрепить за узлом, где лежат данные (в моём случае через `nodeSelector: role: db` на master). Чтобы под мог свободно переезжать между узлами, нужно сетевое хранилище, например NFS, Longhorn или Ceph.

Этап 2. CI в GitHub Actions

Репозиторий и ветка

Работа выполнена в репозитории github.com/fYmious/kubernetes-intro, ветка `lab-6`. Приложение лежит в каталоге `solutions/assist-py/`, манифесты в `k8s/`.

Workflow `.github/workflows/ci.yml`

```yaml
name: ci

on:
  push:
    branches: [main, lab-6]
  workflow_dispatch:

permissions:
  contents: read
  packages: write

env:
  IMAGE: ghcr.io/${{ github.repository }}

jobs:
  test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: solutions/assist-py
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install dependencies
        run: pip install --no-cache-dir -r requirements.txt
      - name: Syntax check
        run: python -m compileall app

  build-and-push:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-qemu-action@v3
      - uses: docker/setup-buildx-action@v3
```

```
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.IMAGE }}
          tags: |
            type=sha,format=short
            type=raw,value=latest
      - uses: docker/build-push-action@v6
        with:
          context: ./solutions/assist-py
          platforms: linux/amd64,linux/arm64
          push: true
          tags: ${{ steps.meta.outputs.tags }}
```

Job `test` ставит зависимости и проверяет синтаксис кода. Job `build-and-push` запускается только после успешного `test` (`needs: test`), собирает образ под `linux/amd64` и `linux/arm64` и публикует его в GHCR с тегами `latest` и коротким SHA коммита.

Запуск пайплайна

```
git add .github/workflows/ci.yml
git commit -m "lab6: add CI"
git push -u origin lab-6
```

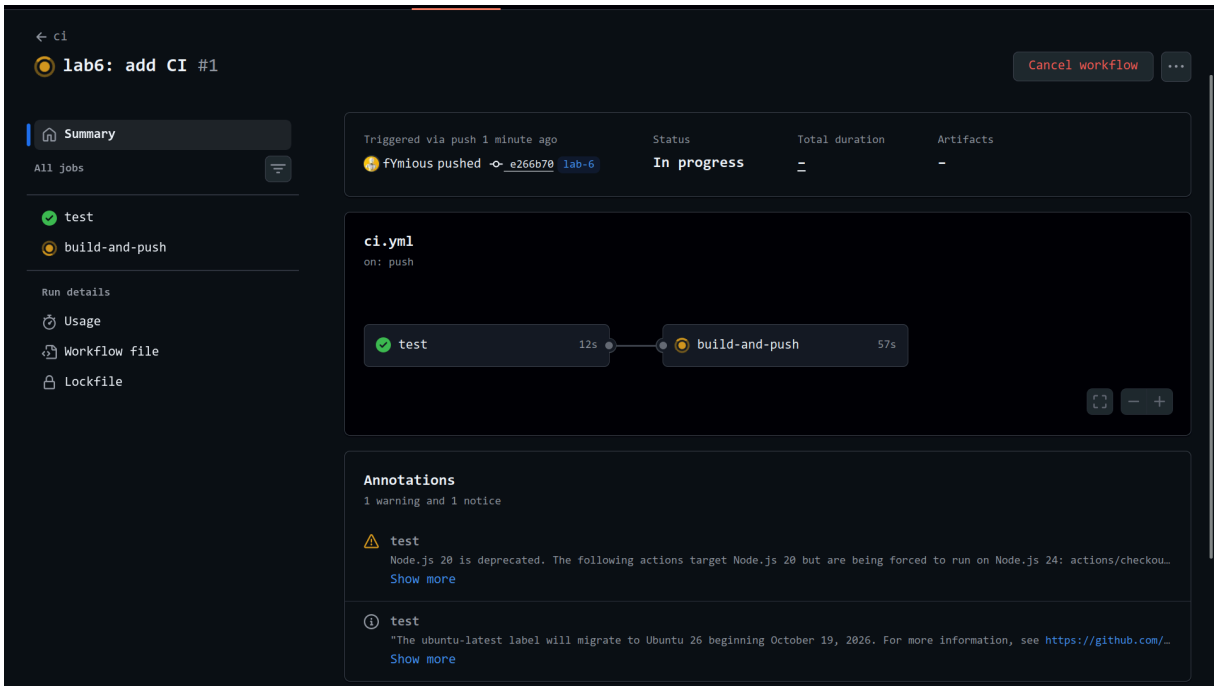


Рисунок 13. Первый запуск пайплайна (`lab6: add CI`): job `test` пройден, идёт сборка образа

Ссылка на вкладку Actions: https://github.com/fYmious/kubernetes-intro/actions. Успешный запуск после изменения приложения (`lab6: update greeting message`, коммит `a1a8834`), оба job зелёные:

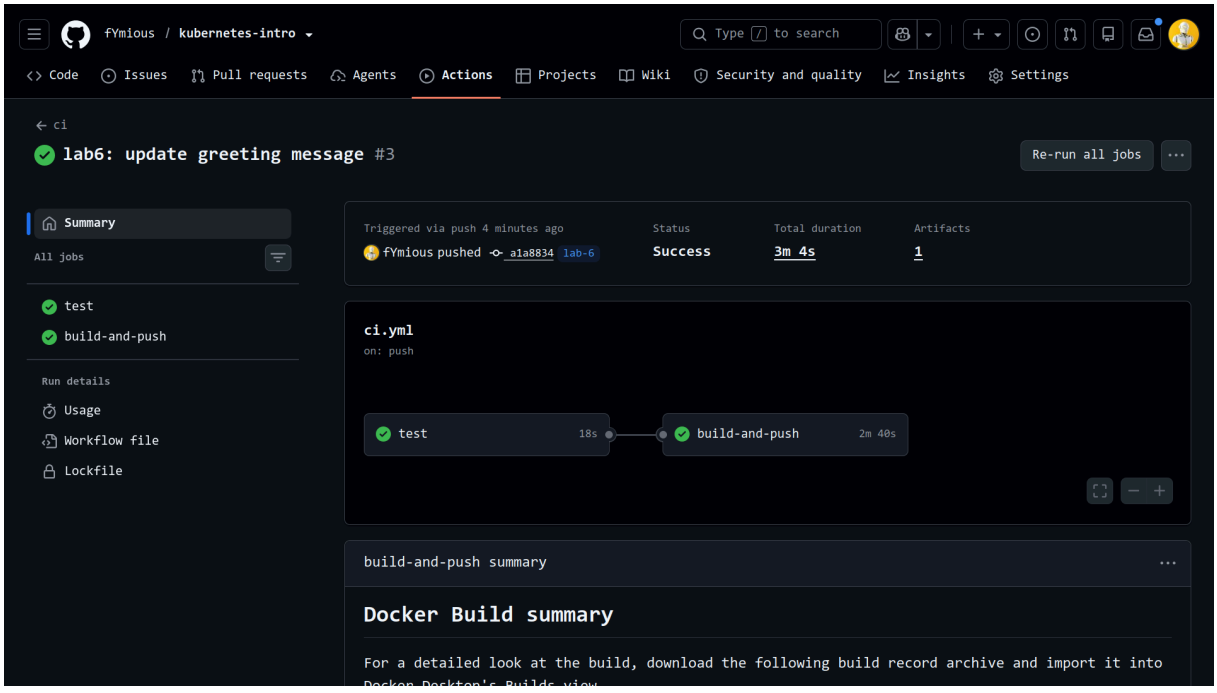


Рисунок 14. Успешный запуск CI: оба job зелёные

Публичный образ

Я сменил видимость пакета на Public и проверил, что образ скачивается без авторизации:

```
sudo k3s crictl pull ghcr.io/fymious/kubernetes-intro:latest
```

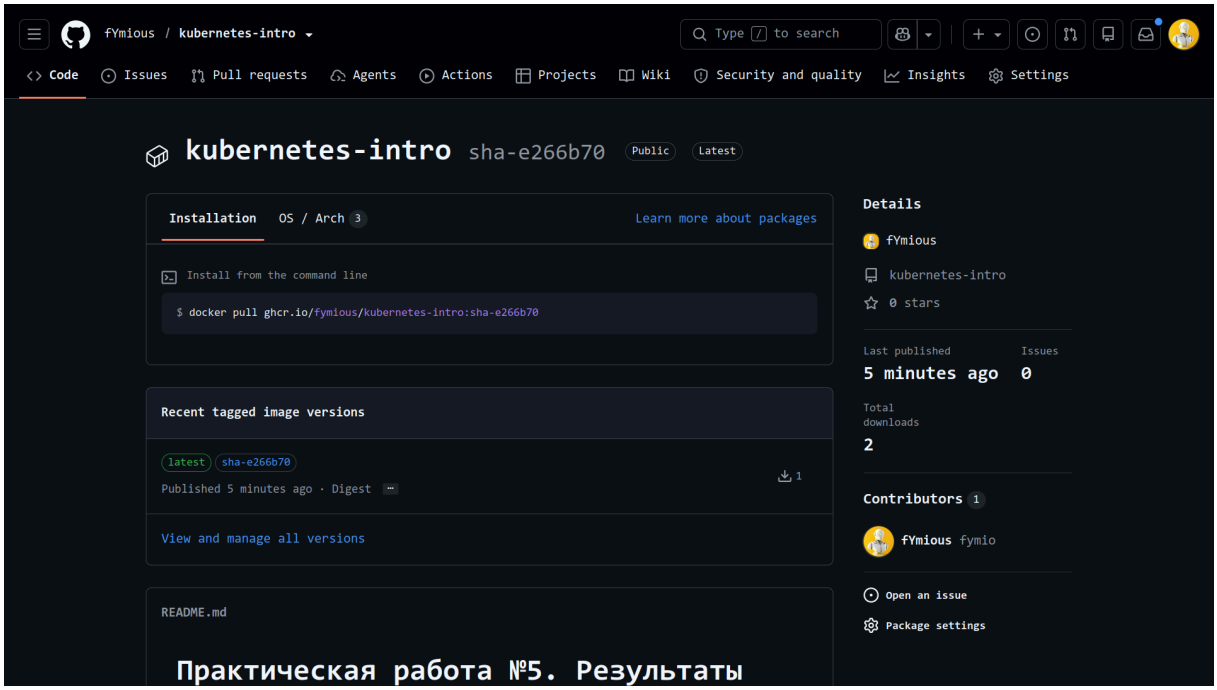


Рисунок 15. Пакет в GHCR: Public, теги `latest` и `sha-e266b70`



Рисунок 16. Скачивание образа без авторизации на master

Этап 3. Развёртывание на двух узлах

Изменения в манифестах

В `k8s/app-deployment.yaml` я переименовал Deployment и метки с префиксом `doschennikov`, указал образ из GHCR с тегом-SHA, поставил `replicas: 4` и добавил `topologySpreadConstraints`, чтобы поды разошлись по обоим узлам. `nodeSelector` для приложения не использовал:

```yaml
spec:
  replicas: 4
  selector:
    matchLabels:
      app: doschennikov-app
  template:
    metadata:
      labels:
        app: doschennikov-app
    spec:
      enableServiceLinks: false
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels:
              app: doschennikov-app
      containers:
        - name: app
          image: ghcr.io/fymious/kubernetes-intro:sha-e266b70
          ports:
            - containerPort: 5527
```

Остальная часть шаблона осталась такой же, как в ЛР №5. Приложение слушает порт 5527.

В `k8s/app-service.yaml` я задал свой NodePort:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: doschennikov-app
spec:
  type: NodePort
  selector:
    app: doschennikov-app
  ports:
    - port: 5527
```

```
    targetPort: 5527
    nodePort: 30097
```

В `k8s/postgres-deployment.yaml` БД закреплена за master:

```
  spec:
    enableServiceLinks: false
    nodeSelector:
      role: db
```

В `k8s/app-configmap.yaml` установлены `APP_OWNER: "doschennikov"` и `POSTGRES_DB: "doschennikov97"`.

Применение манифестов

```
git clone -b lab-6 https://github.com/fYmious/kubernetes-intro.git
cd kubernetes-intro
kubectl apply -f k8s/
kubectl get pods -o wide
```

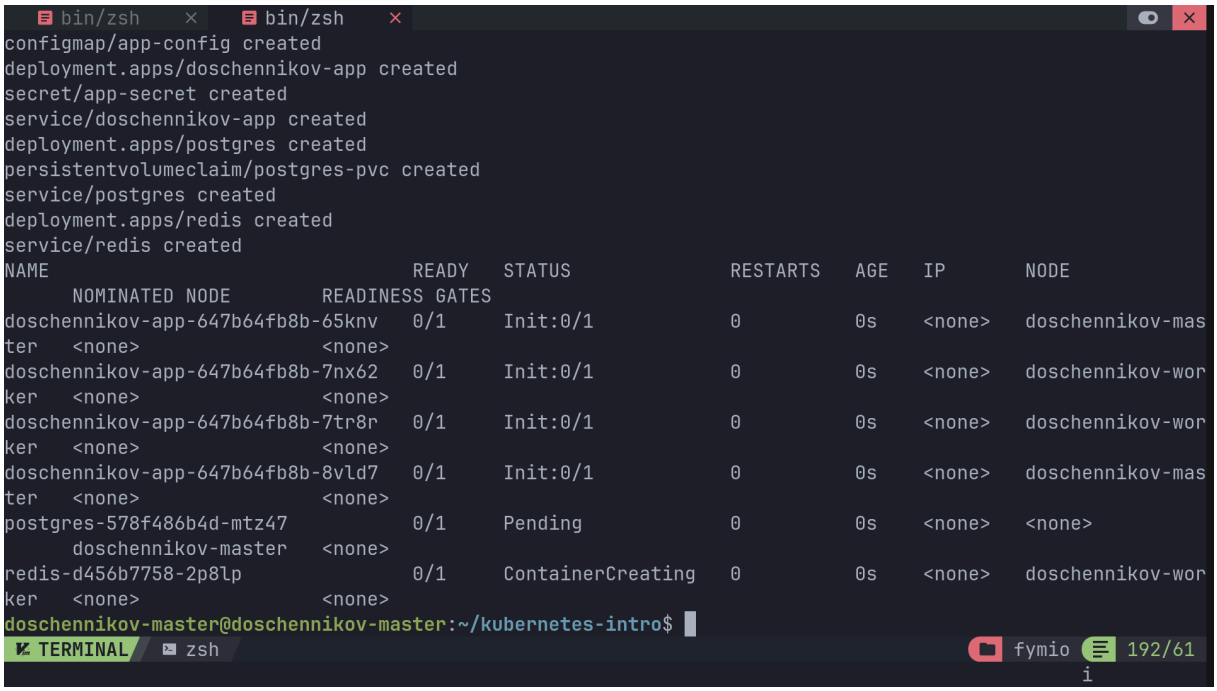


Рисунок 17. `kubectl apply -f k8s/` и начальный запуск подов

```
^Cdoschennikov-master@doschennikov-master:~/kubernetes-intro$ kubectl get pods -o wide -w
NAME                                READY   STATUS    RESTARTS   AGE    IP           NODE
 NOMINATED NODE   READINESS GATES
doschennikov-app-647b64fb8b-65knv   1/1     Running   0          100s   10.42.0.9    doschennikov-master
 <none>           <none>
doschennikov-app-647b64fb8b-7nx62   1/1     Running   0          100s   10.42.1.4    doschennikov-worker
 <none>           <none>
doschennikov-app-647b64fb8b-7tr8r   1/1     Running   0          100s   10.42.1.3    doschennikov-worker
 <none>           <none>
doschennikov-app-647b64fb8b-8vld7   1/1     Running   0          100s   10.42.0.10   doschennikov-master
 <none>           <none>
postgres-578f486b4d-mtz47           1/1     Running   0          100s   10.42.0.12   doschennikov-master
 <none>           <none>
redis-d456b7758-2p8lp               1/1     Running   0          100s   10.42.1.5    doschennikov-worker
 <none>           <none>
```

Рисунок 18. Все поды Running: приложение на master и worker, postgres на master

Поды приложения распределились по два на каждый узел, `postgres` стоит на master (по `nodeSelector`), `redis` попал на worker.

Доступ с обеих машин

NodePort открыт на каждом узле кластера, поэтому приложение отвечает и по адресу master, и по адресу worker:

```
curl http://192.168.122.46:30097/
curl http://192.168.122.217:30097/
curl http://192.168.122.46:30097/health
```

```
doschennikov-master@doschennikov-master:~/kubernetes-intro$ curl http://192.168.122.46:30097/
curl http://192.168.122.217:30097/
curl http://192.168.122.46:30097/health
{"message":"👋 Привет, doschennikov!","service":"personal-assistant","version":"3.0","endpoints":["/me","/
notes","/weather/{city}","/secret/{key}","/cache/stats"],"hostname":"doschennikov-app-647b64fb8b-7nx62","s
erver_time":"2026-10-23T09:34:20.330131+00:00"}{"message":"👋 Привет, doschennikov!","service":"personal-a
ssistant","version":"3.0","endpoints":["/me","/notes","/weather/{city}","/secret/{key}","/cache/stats"],"h
ostname":"doschennikov-app-647b64fb8b-7tr8r","server_time":"2026-10-23T09:34:20.338113+00:00"}{"status":"h
ealthy","owner":"doschennikov","redis":true,"hostname":"doschennikov-app-647b64fb8b-8vld7","server_time":"
2026-10-23T09:34:20.411989+00:00"}doschennikov-master@doschennikov-master:~/kubernetes-intro$
```

Рисунок 19. Ответы `GET /` через оба узла и `GET /health`

Балансировка

```
for i in $(seq 1 10); do curl -s http://192.168.122.217:30097/; echo;
done
```



Рисунок 20. Разные значения `hostname` в ответах: запросы распределяются по подам

В ответах встречаются разные имена подов, то есть Service балансирует запросы между подами на двух узлах.

Кэш и база данных

```
curl http://192.168.122.46:30097/weather/Moscow
curl http://192.168.122.217:30097/weather/Moscow
```

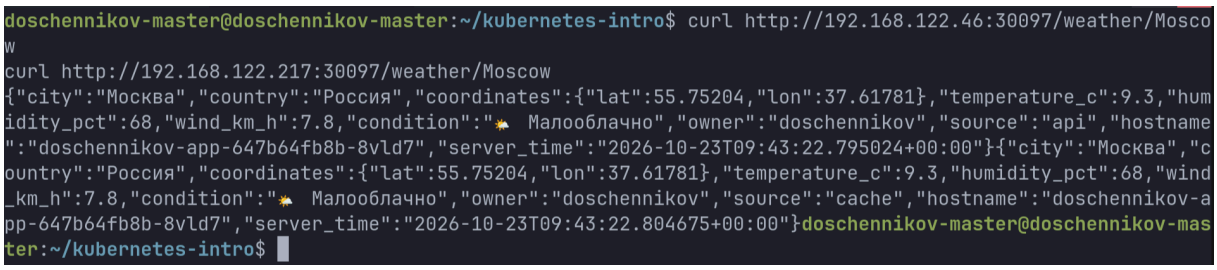


Рисунок 21. Первый запрос `"source": "api"`, второй `"source": "cache"`

Заметка создаётся через master и читается через worker, то есть запрос попадает на другой узел, а данные берутся из общей БД:

```
curl -X POST http://192.168.122.46:30097/notes -H "Content-Type: application/json" -d '{"text":"lab6 note from master doschennikov"}'
curl http://192.168.122.217:30097/notes
```

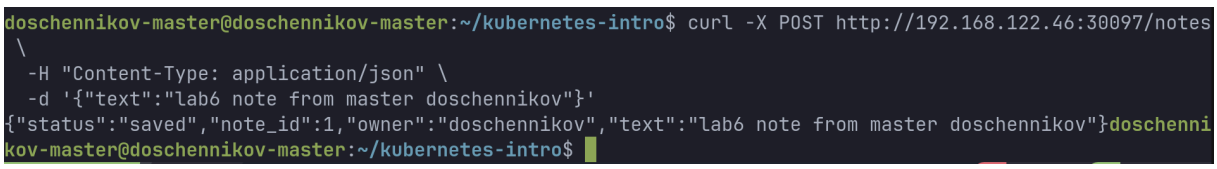


Рисунок 22. `POST /notes` через master



Рисунок 23. `GET /notes` через worker: заметка на месте

Проверка APP_OWNER

Значение `APP_OWNER` приходит через ConfigMap и содержит мою фамилию:

```
kubectl get configmap app-config -o yaml | grep APP_OWNER
```

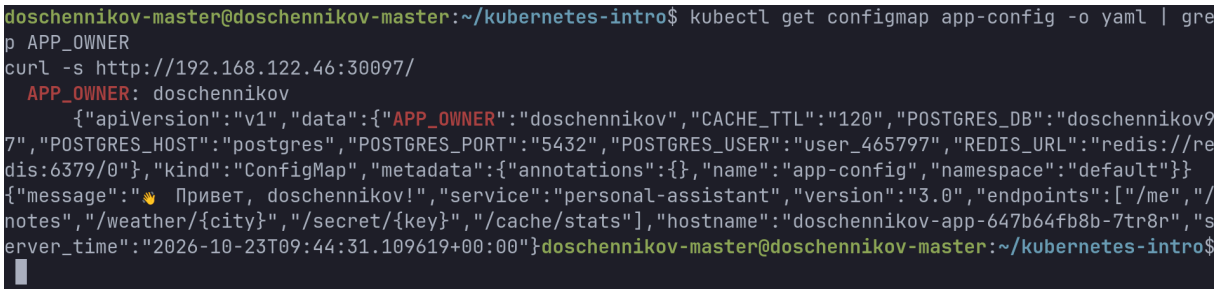


Рисунок 24. `APP_OWNER` в ConfigMap и ответ `GET /`

Этап 4. Простой CD: обновление приложения

Изменение приложения

Я изменил в `GET /` текст приветствия (`v2 (lab6 rolling update)`) и версию (`3.1`), сделал коммит `lab6: update greeting message` и отправил его в `lab-6`. Пайплайн отработал успешно (см. запуск CI выше), новый образ получил тег `sha-a1a8834`.

Rolling update

```
kubectl get deployment doschennikov-app -o
jsonpath='{.spec.template.spec.containers[*].name}'
kubectl set image deployment/doschennikov-app app=ghcr.io/fymious/
kubernetes-intro:sha-a1a8834
kubectl rollout status deployment/doschennikov-app
kubectl get pods -o wide
```

Имя контейнера в манифесте `app`. Во время обновления в соседнем терминале я наблюдал за подами (`kubectl get pods -o wide -w`). Kubernetes создаёт новые поды и постепенно удаляет старые, не останавливая сервис целиком.

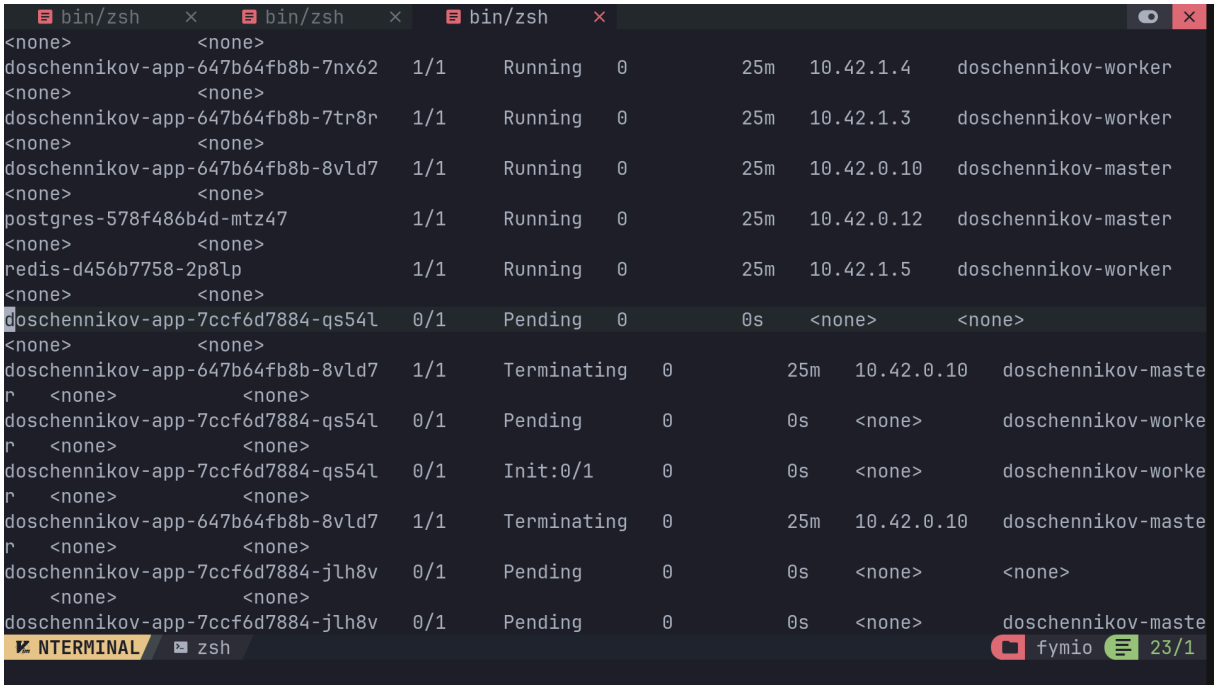


Рисунок 25. Наблюдение за подами: новые поды создаются, старые переходят в Terminating

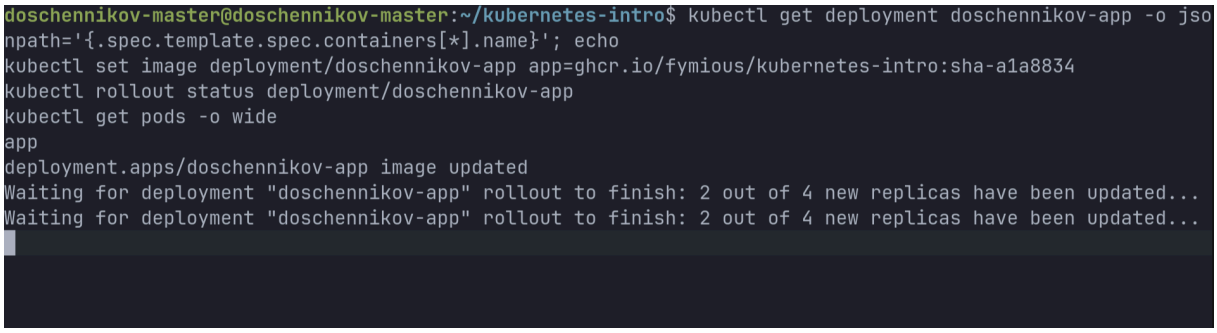


Рисунок 26. `kubectl set image` и ход `rollout status`

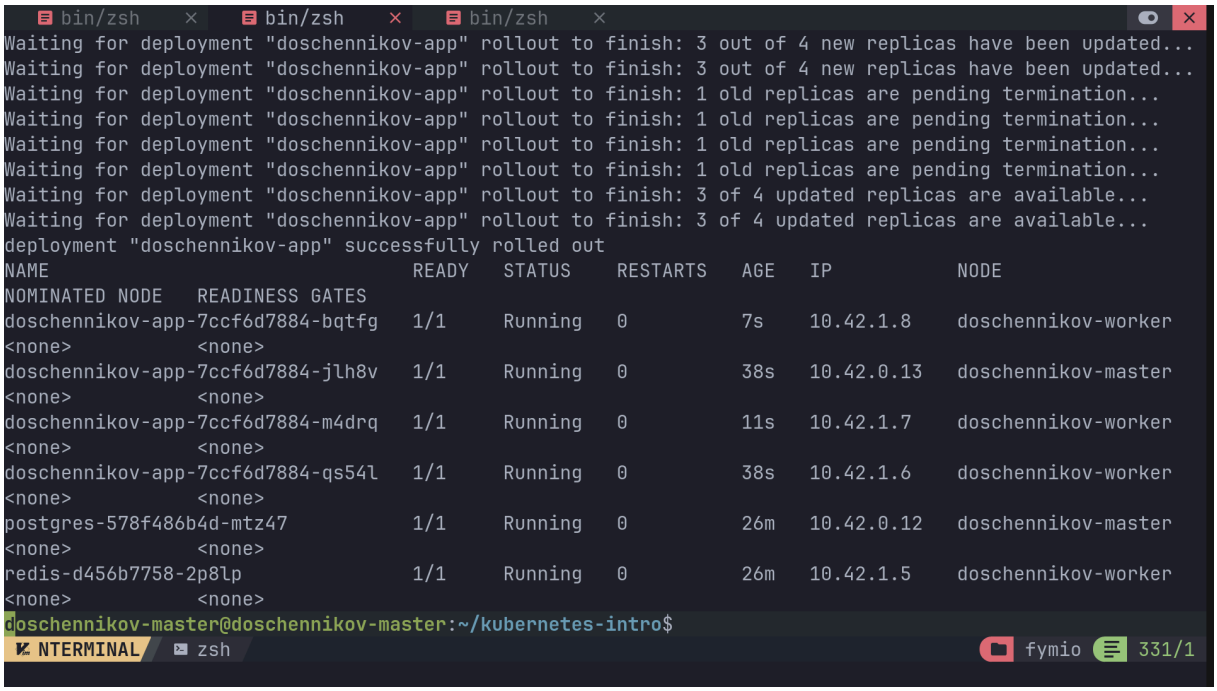


Рисунок 27. Обновление завершено: `successfully rolled out`, новые поды Running

После обновления приложение отвечает новым текстом:

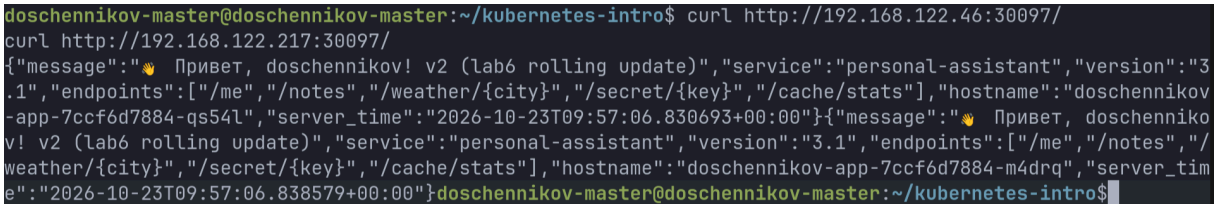


Рисунок 28. `GET /` возвращает `v2 (lab6 rolling update)` и версию 3.1

История и откат

```
kubectl rollout history deployment/doschennikov-app
kubectl rollout undo deployment/doschennikov-app
kubectl rollout status deployment/doschennikov-app
curl http://192.168.122.46:30097/
```

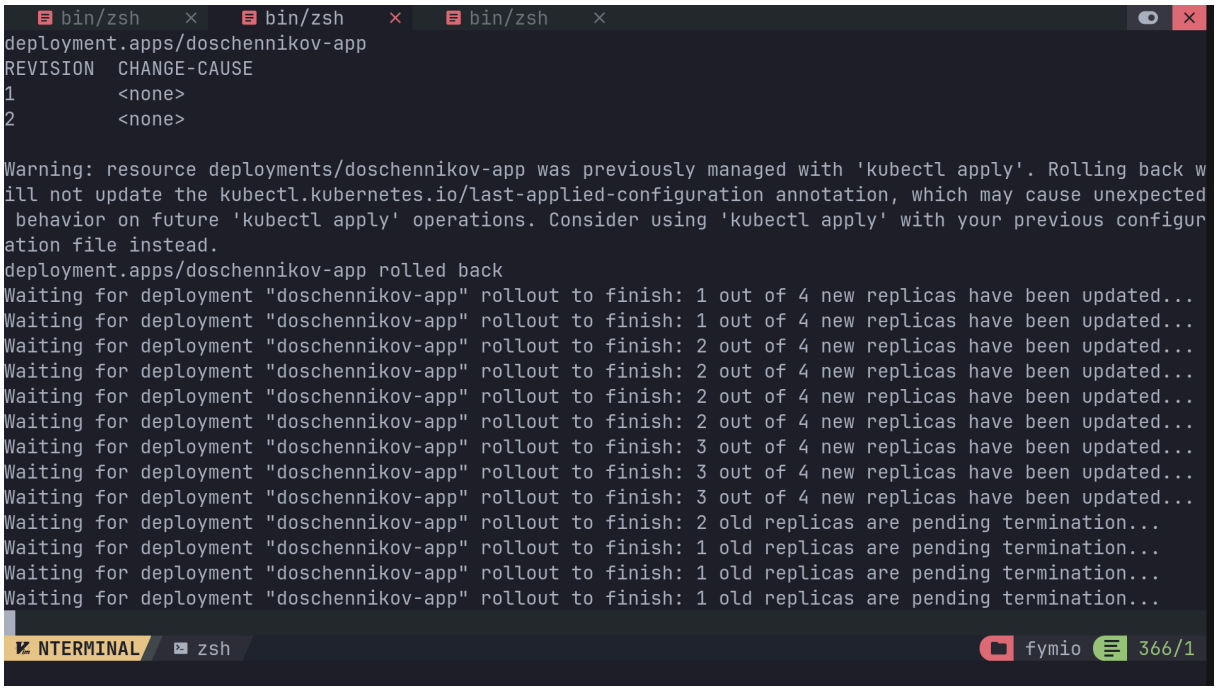


Рисунок 29. `rollout history` (ревизии 1 и 2) и `rollout undo`



Рисунок 30. Откат завершён, в ответе снова старое приветствие и версия 3.0

Для финального состояния я снова выкатил версию `sha-a1a8834`:

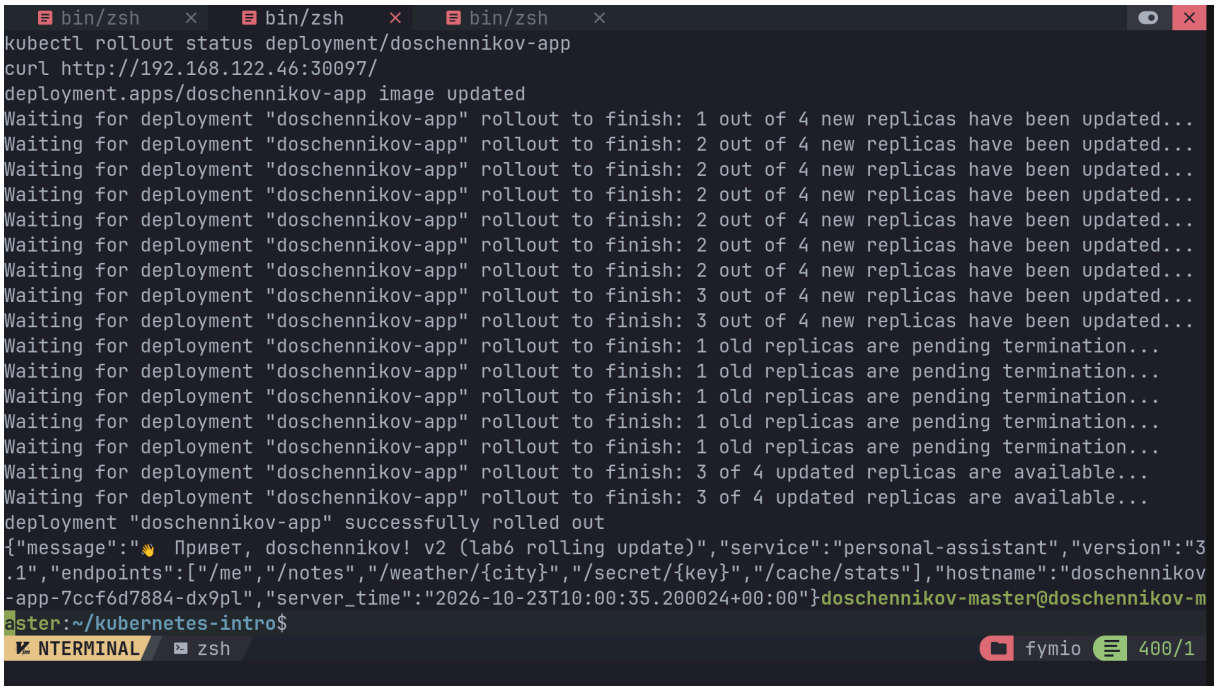


Рисунок 31. Повторная выкатка `sha-a1a8834`: в ответе снова `v2 (lab6 rolling update)`

- Почему в манифесте для production лучше использовать тег с SHA (или digest), а не latest?

Тег `latest` изменяемый, он указывает на разные образы в разные моменты времени, и по манифесту невозможно понять, какая именно версия работает. Если под перезапустится на другом узле, он может скачать уже новый образ, а узлы с закэшированным образом останутся на старом, и версии в кластере разойдутся. Откат тоже теряет смысл, потому что «предыдущий `latest`» нигде не зафиксирован. Тег с SHA коммита неизменяемо привязан к конкретной сборке: деплой воспроизводим, `kubectl rollout undo` возвращает именно прошлую версию, а версию в кластере можно сопоставить с кодом в Git.

- манифест в Git соответствует кластеру

Я обновил тег в `k8s/app-deployment.yaml` отдельным коммитом, затем на master выполнил `git pull` и `kubectl apply -f k8s/`:

```
sed -i 's|kubernetes-intro:sha-e266b70|kubernetes-intro:sha-a1a8834|'
k8s/app-deployment.yaml
git add k8s/app-deployment.yaml
git commit -m "lab6: bump image tag to sha-a1a8834"
git push
```

```
❯ sed -i 's|kubernetes-intro:sha-e266b70|kubernetes-intro:sha-a1a8834|'
k8s/app-deployment.yaml
grep image: k8s/app-deployment.yaml
git add k8s/app-deployment.yaml
git commit -m "lab6: bump image tag to sha-a1a8834"
git push
          image: postgres:15
          image: ghcr.io/fymious/kubernetes-intro:sha-a1a8834
[lab-6 6395f48] lab6: bump image tag to sha-a1a8834
 1 file changed, 1 insertion(+), 1 deletion(-)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 374 bytes | 374.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/fYmious/kubernetes-intro.git
   a1a8834..6395f48  lab-6 -> lab-6
```

Рисунок 32. Коммит с новым тегом образа в манифесте

```
bin/zsh   bin/zsh   bin/zsh
kubectl get deployment doschennikov-app -o jsonpath='{.spec.template.spec.containers[0].image}'; echo
remote: Enumerating objects: 17, done.
remote: Counting objects: 100% (17/17), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 10 (delta 7), reused 10 (delta 7), pack-reused 0 (from 0)
Unpacking objects: 100% (10/10), 806 bytes | 403.00 KiB/s, done.
From https://github.com/fYmious/kubernetes-intro
   f55c66f..6395f48  lab-6      -> origin/lab-6
Updating f55c66f..6395f48
Fast-forward
 k8s/app-deployment.yaml         | 2 +-
 solutions/assist-py/app/main.py | 4 ++--
 2 files changed, 3 insertions(+), 3 deletions(-)
configmap/app-config unchanged
deployment.apps/doschennikov-app configured
secret/app-secret configured
service/doschennikov-app unchanged
deployment.apps/postgres unchanged
persistentvolumeclaim/postgres-pvc unchanged
service/postgres unchanged
deployment.apps/redis unchanged
service/redis unchanged
ghcr.io/fymious/kubernetes-intro:sha-a1a8834
doschennikov-master@doschennikov-master:~/kubernetes-intro$
NTERMINAL  zsh                                   fymio  425/60
```

Рисунок 33. `git pull`, `kubectl apply -f k8s/` и образ в кластере `sha-a1a8834`

Образ в Deployment совпадает с тегом в манифесте из Git. Для `doschennikov-app` `apply` вывел `configured`, а не `unchanged`: после ручных `set image` и `rollout undo` аннотация `last-applied-configuration` отличалась от файла, и `apply` её обновил. Остальные ресурсы остались `unchanged`.

Этап 5. Отказоустойчивость и финальный артефакт

Отключение worker

Я отключил worker при помощи

```
sudo systemctl stop k3s-agent
```

и через некоторое время статус worker стал `NotReady`. Затем я вернул worker обратно:

```
sudo systemctl start k3s-agent
```

Финальный артефакт

Финальная команда выполнена на master, когда оба узла были Ready, поды работали на обоих узлах, а в Deployment стоял образ с тегом-SHA:

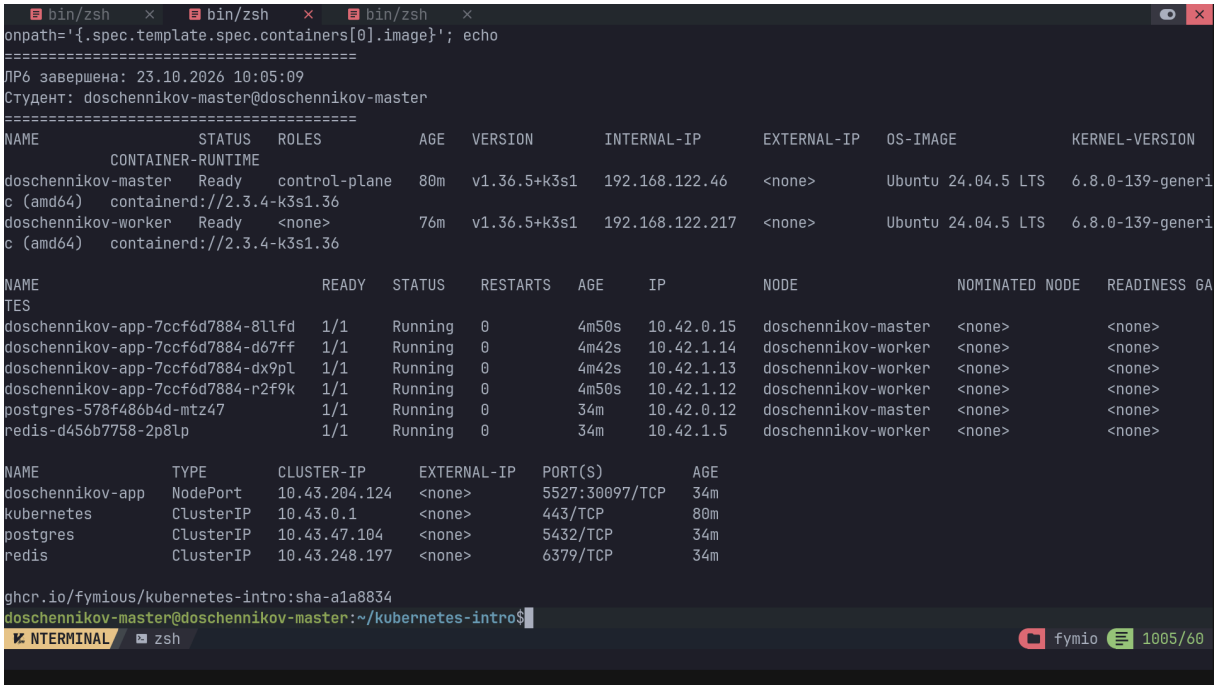


Рисунок 34. Финальный артефакт ЛР6